

Reflection and Self-Directed Learning in AI-Supported Programming Education

Investigating the Impact of Reflective Prompts on Student Engagement
and Problem-Solving in Automated Code Assessment

Bachelor's Thesis

submitted in conformity with the requirements for the degree of

Bachelor of Science in Engineering (BSc)

Bachelor's degree programme **Software Design and Cloud
Computing**

FH JOANNEUM (University of Applied Sciences), Kapfenberg

Supervisor: Johannes Feiner

Submitted by: Paria Nikparsa

May 2026

Abstract

This thesis examines a limitation of Automated Code Assessment (ACA) systems in programming education. Although ACA systems are effective at evaluating functional correctness through automated grading, they provide limited insight into students' problem-solving strategies, revision behavior after feedback, and the use of AI-assisted tools during the coding process. To improve process visibility, a mandatory reflection module was integrated directly into the submission workflow of the ACA system. The implemented prototype combines asynchronous automated grading with structured and open-ended reflection capture at submission time. The system includes attempt-aware prompts (first attempt: learning and difficulties; second attempt: revision rationale), role-based access control, one-reflection-per-submission validation, and teacher-side coherence assessment. This design links technical outputs (status, score, passed tests) with process-oriented indicators (strategy, AI-use self-report, revision explanation) through shared identifiers, enabling traceable mixed-method analysis.

The evaluation of this research was conducted in two parts: first, a prototype and evidence-readiness assessment using structured scenario-based testing; second, a small-scale exploratory user study aimed at collecting initial feedback on the experience of using the system. The results indicate that the end-to-end workflow operates reliably, required-field and ownership constraints are correctly enforced, and technical and reflective records remain consistently linked for subsequent analysis. In addition, preliminary findings from the user study suggest that the reflection-enhanced workflow may encourage more process-oriented engagement; however, due to the limited scale of the study, these findings cannot be generalized.

Zusammenfassung

Diese Arbeit untersucht eine Einschränkung von Automated Code Assessment (ACA)-Systemen in der Programmierausbildung. Obwohl ACA-Systeme die funktionale Korrektheit von Lösungen durch automatisierte Bewertung zuverlässig überprüfen können, bieten sie nur begrenzte Einblicke in die Problemlösungsstrategien der Studierenden, ihr Überarbeitungsverhalten nach Feedback sowie die Nutzung KI-gestützter Werkzeuge während des Programmierprozesses. Um die Transparenz des Lösungsprozesses zu verbessern, wurde ein verpflichtendes Reflexionsmodul direkt in den Abgabe-Workflow des ACA-Systems integriert.

Der implementierte Prototyp kombiniert asynchrone automatisierte Bewertung mit der Erfassung strukturierter und offener Reflexionen zum Zeitpunkt der Abgabe. Das System umfasst versuchsabhängige Eingabeaufforderungen (erster Versuch: Lernerfahrungen und Schwierigkeiten; zweiter Versuch: Begründung der Überarbeitung), rollenbasierte Zugriffskontrolle, eine Validierung „eine Reflexion pro Abgabe“ sowie eine lehrendenseitige Kohärenzbewertung. Dieses Design verbindet technische Ergebnisse (Status, Punktzahl, bestandene Tests) über gemeinsame Identifikatoren mit prozessorientierten Indikatoren (Strategie, Selbstauskunft zur KI-Nutzung, Erklärung der Überarbeitung) und ermöglicht dadurch eine nachvollziehbare Mixed-Methods-Analyse.

Die Evaluation dieser Arbeit wurde in zwei Teilen durchgeführt: Erstens erfolgte eine Prototyp- und Evidence-Readiness-Evaluation mithilfe strukturierter szenariobasierter Tests; zweitens wurde eine explorative Nutzerstudie im kleinen Maßstab durchgeführt, um erstes Feedback zur Nutzungserfahrung des Systems zu sammeln. Die Ergebnisse zeigen, dass der End-to-End-Workflow zuverlässig funktioniert, verpflichtende Felder sowie Eigentumsbeschränkungen korrekt durchgesetzt werden und technische sowie reflexive Daten konsistent für spätere Analysen verknüpft bleiben. Darüber hinaus deuten erste Ergebnisse der Nutzerstudie darauf hin, dass der reflexionsgestützte Workflow eine stärker prozessorientierte Beteiligung fördern könnte; aufgrund des begrenzten Umfangs der Studie lassen sich diese Ergebnisse jedoch nicht verallgemeinern.

Acknowledgement

To Ehsan—your belief has been my guiding light.

Contents

List of Figures	iii
List of Tables	iv
1 Introduction	1
1.1 Problem Statement	2
1.2 Research Gap	2
1.3 Research Questions	3
1.4 Hypothesis	4
1.5 Method	5
2 Related Work	7
3 Concept	11
3.1 Conceptual Workflow and Data Logic	11
3.2 Positioning Against Alternatives	12
3.3 Expected Value, Trade-offs, and Requirements	13
4 Implementation	14
4.1 System Architecture and Component Responsibilities	14
4.2 Submission Lifecycle and Asynchronous Grading	15
4.3 Reflection Module: Workflow-Coupled Design	16
4.4 Reflection Data Schema and Validation Logic	18
4.5 Teacher Review and Coherence Assessment	19
4.6 Data Model and Traceability for Analysis	19
4.7 UI Implementation Decisions and Usability Trade-offs	19
4.8 Difficult and Innovative Aspects of the Prototype	20
4.9 Implementation Outcome and Readiness	21
5 Evaluation	22
5.1 Exploratory User Study with Target Groups	27
6 Conclusion and Outlook	31
6.1 Interpretation of Results	32

6.2	Limitations	32
6.3	Outlook and Future Work	33
A	Survey Response Tables	34
A.1	Control Group: Full Item-Level Responses	34
A.2	Experimental Group: Full Item-Level Responses	34
	Acronyms	43
	Bibliography	44

List of Figures

4.1	System architecture of the ACA-Reflection prototype, showing front-end, backend, runner, and data storage, including request/callback directions.	15
4.2	Entity relationship view of users, assignments, submissions, results, and reflections, including key linkage fields.	20
5.1	Reflection form completion gating in the student submission flow. With incomplete inputs (top), the Submit reflection action remains inactive; after all required open-ended and structured fields are completed (bottom), the button becomes active and reflection submission is enabled.	23
5.2	Confirmation dialog shown when a student attempts to close the mandatory reflection form before completion. The warning makes clear that submission will not be finalized and no record will appear in student or teacher dashboards unless the reflection process is completed.	24
5.3	End-to-end reflection feedback loop across teacher and student views. In the Teacher Center (top), instructors review student reflections and record structured assessment comments; in My Submissions (bottom), students see the same evaluation outcome and feedback, ensuring transparent reflection-based guidance after grading.	25
5.4	Second-attempt reflection form capturing post-feedback changes, improvement rationale, and remaining uncertainties.	26

List of Tables

5.1	Scenario-based evaluation overview	27
5.2	Stored variables and derived indicators for later analysis	28
5.3	Participant Demographics	29
5.4	Survey questions for the control group	29
5.5	Reflection prompts used in the experimental group by submission attempt	30
A.1.1	Control Group Survey Responses (Participant 1)	35
A.1.2	Control Group Survey Responses (Participant 2)	35
A.1.3	Control Group Survey Responses (Participant 3)	36
A.1.4	Control Group Survey Responses (Participant 4)	36
A.1.5	Control Group Survey Responses (Participant 5)	37
A.2.1	Experimental Group Responses (Participant 1)	38
A.2.2	Experimental Group Responses (Participant 2)	39
A.2.3	Experimental Group Responses (Participant 3)	40
A.2.4	Experimental Group Responses (Participant 4)	41
A.2.5	Experimental Group Responses (Participant 5)	42

Introduction

In contemporary education, programming is taught at scale: large lecture classes, blended learning environments, and online platforms mean that instructors cannot manually review every line of code submitted by every student on each attempt. Automated Code Assessment (ACA) systems address this limitation by executing a suite of tests on student submissions and providing rapid, consistent feedback on program correctness, and in some designs, additional signals such as coding style or complexity. In educational practice, ACA is often experienced as the “engine” behind weekly exercises and assignments: students submit their solution, the system runs tests, and within seconds or minutes a result or score is displayed.

This strength - namely scalable and repeatable verification of program behavior - simultaneously introduces a limitation when learning is considered beyond merely producing a correct output. Competent programmers must be able to debug, conduct code reviews, justify their design decisions, and increasingly collaborate with tools that generate or suggest code. In particular, tools based on large language models (LLMs) and other AI-driven code generation systems are now widely accessible. Students can obtain complete solutions or substantial portions of code with minimal friction. Policy responses range from outright prohibition (often unenforceable) to unrestricted use accompanied by guidelines. A complementary educational question is whether the assessment environment itself can encourage learners to engage actively with problems and explicitly reflect on how they use assistance—including AI—rather than delegating the cognitive process entirely.

Reflection in education refers to the conscious attention to one’s own thought processes, strategies, and difficulties. Metacognition refers to awareness and regulation of cognitive processes and plays a central role in self-regulated learning, where learners

set goals, monitor their progress, and adjust their strategies. Short reflective prompts, placed at meaningful points during task execution, provide a lightweight way to stimulate these processes without requiring a separate course in learning theory.

1.1 Problem Statement

The central problem of this research is the misalignment between what Automated Code Assessment (ACA) systems effectively measure and what programming courses are expected to cultivate in students in the era of artificial intelligence (AI) tools. While ACA systems primarily focus on outcome-based evaluation—such as pass/fail results and test case correctness—and, in some cases, technical indicators such as output validity or code quality, they generally fail to capture the process-oriented dimensions of learning. These include how students approach a problem, their level of conceptual understanding, points of confusion, and their use of external tools such as large language models (LLMs).

In the context of the widespread availability of AI-based code generation tools, this limitation becomes increasingly significant. Students can obtain complete or near-complete solutions with minimal cognitive engagement, while ACA systems continue to evaluate only the final correctness of submissions. This creates a gap between “what is assessed” and “what should be learned,” a gap that is particularly critical in relation to self-regulated learning, critical thinking, and responsible use of AI tools.

Although ACA systems offer important advantages such as scalability, reduced instructor workload, rapid feedback, and increased fairness through uniform evaluation criteria, they also exhibit significant limitations. First, there is a lack of process visibility; final outcomes (pass/fail) provide no information about the solution path, depth of understanding, or reliance on external assistance. Second, there are cognitive and motivational risks associated with AI usage, as students may uncritically accept AI-generated solutions. Third, reflective activities are often implemented as separate components outside the main problem-solving workflow, reducing their temporal relevance and educational effectiveness.

1.2 Research Gap

Based on the above considerations, the main research gap lies in the design and evaluation of an approach that can simultaneously:

- preserve the benefits of automated testing systems (e.g., scalability and rapid

feedback);

- integrate a lightweight, in-process reflection layer into the problem-solving workflow;
- enable the capture and measurement of learning behaviors, including the use of AI tools;
- provide a basis for empirical comparison between different versions of the assessment system (with and without the reflection layer).

1.3 Research Questions

This study investigates the extent to which mandatory reflective prompts about the learning process and the use of artificial intelligence tools, integrated into an Automated Code Assessment (ACA) system, influence self-regulated learning and independent problem-solving abilities in introductory programming exercises.

1.3.1 Operationalization of Variables

In this study, two main constructs are examined. Since these constructs are not directly observable, they are operationalized through a combination of behavioral data recorded by the ACA system and students' self-reported data.

1.3.1.1 Self-Regulated Learning

Self-regulated learning refers to students' ability to actively manage their own learning process, including planning, monitoring, and adjusting their performance during problem-solving. This construct is operationalized through behavioral indicators extracted from the ACA system. These indicators include the time to first submission, the number of submission attempts, the success rate on the first attempt, the time intervals between consecutive attempts, and the extent of modifications made across successive code versions. Collectively, these measures reflect the level of active engagement, persistence, and iterative refinement demonstrated by students during the problem-solving process.

1.3.1.2 Independent Problem-Solving

Independent problem-solving refers to the degree to which students rely on their own cognitive effort in contrast to external assistance, particularly AI-based tools. This

variable is measured through a combination of self-reported data and students' reflective responses. Specifically, it includes the reported extent of AI tool usage (e.g., ChatGPT), students' perceived individual contribution to the solution, their ability to explain and justify the logic of their code, and the quality of their reflections on the problem-solving process and how they arrived at their final solution. Together, these indicators provide an overall picture of the degree of genuine independence in students' problem-solving behavior.

1.4 Hypothesis

1.4.1 Central Hypothesis

The integration of structured reflective activities into an Automated Code Assessment (ACA) workflow leads to increased active engagement with programming tasks and enhanced self-regulated learning behaviors, while simultaneously reducing passive reliance on artificial intelligence tools, compared to a condition without the reflection module.

1.4.2 Proposed Approach

This study examines a socio-technical intervention aimed at embedding reflection within the natural problem-solving workflow of an Automated Code Assessment (ACA) environment. The core objective is to position reflection as an integral part of the learning process rather than a separate, peripheral activity.

To this end, a reflection module is implemented within the ACA user interface. This module is triggered at predefined points during the problem-solving process and requires students to complete short prompts regarding their problem-solving approach, encountered difficulties, and the extent of their use of artificial intelligence tools. The resulting reflection data are stored at the level of each code submission and linked one-to-one with individual attempts, enabling fine-grained behavioral analysis.

In addition, a minimal instructor-side monitoring mechanism is provided to allow basic review of open-ended responses. The evaluation design follows a parallel-group setup, consisting of a control group and an experimental group, both working on identical programming tasks under comparable conditions. The key difference lies in the presence of the reflection module, which is only activated for the experimental group.

1.4.3 Analytical Framework

The primary hypothesis is directional. It is expected that the inclusion of structured reflection will increase behavioral indicators associated with active engagement, such as the number of attempts, the extent of code revisions, and persistence across iterations. Furthermore, a reduction in self-reported reliance on artificial intelligence tools is anticipated in the experimental group.

At the same time, certain variables, particularly self-reported AI usage, are treated as exploratory due to the potential for reporting bias inherent in self-assessment data.

Given the quasi-experimental nature of the design, potential confounding factors such as individual differences in prior programming experience, baseline skill level, and the absence of full randomization are explicitly acknowledged. Accordingly, statistical analysis will be complemented by a critical interpretation of these limitations and their implications for the validity and generalizability of the findings.

1.5 Method

This study follows a structured approach that integrates a literature-informed theoretical foundation, prototype development, controlled comparison of system configurations, and systematic analysis of results. This approach ensures that both the technical feasibility of the proposed solution and its analytical capabilities are rigorously examined.

The conceptual foundation of the study is grounded in three related research domains: automated code assessment, artificial intelligence in programming education, and reflective learning processes.

To operationalize the proposed idea, a prototype of an Automated Code Assessment (ACA) system was developed. The system is designed as a multi-component architecture, comprising a web-based user interface, a backend service layer, and an isolated execution environment for running automated tests. It supports the full exercise management cycle, including submission, automated evaluation, and feedback delivery. In addition, a reflection module is integrated into the system, which is activated immediately after a solution is submitted.

The experimental distinction in this study is established through two different configurations of the same system. In the baseline configuration, the process follows the conventional ACA workflow without reflection. In the enhanced configuration, a mandatory reflection stage is introduced, requiring students to report on their learning

process, experienced difficulties, and the extent of their use of external resources. The reflection prompts are designed to distinguish between different stages of problem-solving, including the initial attempt and subsequent revisions.

This study adopts a quasi-experimental design in which two groups are compared under similar conditions, differing only in one key aspect: the presence or absence of the reflection module.

Related Work

Recent research in programming education indicates that generative AI introduces a persistent tension between observable performance gains and the quality of underlying learning processes. Controlled classroom studies show that AI assistance can significantly improve programming outcomes and task completion efficiency; however, these same studies also report substantial uncritical acceptance of erroneous AI-generated output, including direct incorporation of incorrect responses into student submissions (Akçapınar et al., 2024). Broader empirical syntheses similarly identify ChatGPT as a widely adopted on-demand support tool for explanation, debugging, and code generation, while simultaneously highlighting risks such as reduced higher-order cognitive engagement, weakened collaborative learning behaviors, and increased academic-integrity concerns when pedagogical guidance is absent (Albadarin et al., 2024).

Complementary work on LLM-based feedback generation further demonstrates the potential of AI systems to provide scalable and personalized formative feedback in programming education. Nevertheless, existing studies continue to report limitations in feedback reliability, internal consistency, and pedagogical suitability, particularly for novice learners who may lack the expertise required to critically evaluate generated responses (Azaiz et al., 2024; Jukiewicz, 2024; Messer et al., 2024). Collectively, these findings suggest that correctness-oriented automated assessment alone is insufficient for supporting durable learning and should instead be coupled with mechanisms that capture learners' reasoning, revision processes, and interaction with external AI assistance.

Within computing education, AI-based code generation is increasingly reshaping introductory programming and its assessment ecology. *Becker et al.* argue that while generative AI tools may improve productivity and broaden accessibility, they also cre-

ate methodological challenges for learning-oriented assessment by encouraging overreliance on generated solutions and reducing visibility into students' underlying reasoning and problem-solving processes (Becker et al., 2023). At the system level, systematic reviews of automated code assessment (ACA) platforms show that contemporary systems are highly optimized for scalability and execution-based evaluation, yet remain limited in their ability to provide pedagogically rich feedback that reflects learners' cognitive and metacognitive processes beyond score production (Comb  fis, 2022). Current ACA infrastructures therefore exhibit a recurring pattern: strong operational efficiency in pass/fail evaluation and grading, but comparatively limited support for observing reflective thinking, revision behavior, and AI reliance during problem solving.

A growing body of intervention-oriented research suggests that structured reflection and scaffolding can mitigate some of these limitations. *Choi et al.* demonstrate that integrating reflection prompts with automated hints produces significantly stronger immediate and delayed transfer-task performance than hints alone, while also increasing perceived learning without negatively affecting learner enjoyment (Choi et al., 2023). Related studies further indicate that AI-supported pair programming can improve motivation, reduce programming anxiety, and enhance performance, although human–human collaboration still appears stronger in terms of perceived social presence and collaborative quality (Fan et al., 2025). Large mixed-method investigations additionally identify a perception–performance gap in which students frequently report increased confidence and perceived utility from AI assistance without corresponding gains in measurable learning outcomes (Gambo et al., 2026).

Across these studies, a consistent pattern emerges: the educational impact of AI depends less on tool availability itself than on instructional design and patterns of learner interaction. Rapid-review syntheses report substantial instructional affordances for tutoring, feedback, and material support, yet simultaneously emphasize risks such as overreliance, superficial task completion, and diminished critical engagement when AI use remains unguided (Garcia, 2025). Classroom interaction studies similarly show that explicit scaffolds, including prompting instruction, verification protocols, and guided AI-use workflows, can shift learners toward more productive interaction behaviors associated with improved outcomes (G  ner et al., 2025).

This instructional-design perspective is reinforced by evidence showing that ChatGPT responses are often prompt-sensitive and error-prone, that AI-assisted grading systems still require human oversight despite reasonable alignment with instructor judgments, and that specialized methods are increasingly necessary to identify AI-

generated submissions where traditional plagiarism-detection approaches become less effective (Hoq et al., 2024; Joshi et al., 2024; Jukiewicz, 2024). At the same time, studies report stronger gains in higher-order thinking, self-efficacy, and implementation quality when AI support is embedded within reflective workflows that require learners to explain, verify, and revise their own work (Lee et al., 2025; Sun et al., 2024; Xie et al., 2023).

Recent evidence further indicates that the educational benefits of AI are conditional rather than uniform across learners and contexts. Although stronger AI models demonstrate improved tutoring and programming-support capabilities, they do not eliminate the need for pedagogical oversight (Phung et al., 2023; Prather et al., 2024). Novice learners remain particularly vulnerable to metacognitive errors and overconfidence, whereas students with stronger prior preparation tend to benefit more consistently from AI-supported workflows (Prather et al., 2024; Suh et al., 2025). Consequently, reported gains in efficiency and assistance coexist with concerns regarding creativity, code-quality practices, and collaborative learning when AI use becomes uncritical or excessively automated (Stoyanova et al., 2025).

Process-oriented studies provide additional support for integrating reflection and structured guidance into programming education. Research on AI-agent-supported collaborative learning environments reports measurable improvements in learning achievement, self-efficacy, and learning interest, alongside reduced mental effort, suggesting that carefully orchestrated AI support can strengthen both cognitive and affective outcomes (H. Wang et al., 2025). Related classroom process analyses of ChatGPT-facilitated programming also show meaningful shifts in programming behavior, with mixed effects on performance and strong effects on learner perceptions (Sun et al., 2024). Replay-based reflection interventions likewise show that revisiting prior programming activity can improve novices' attention to planning and prompt interpretation, although benefits vary depending on the quality of instructional support provided during reflection (Xie et al., 2023). Similar findings in primary programming education demonstrate that ChatGPT-supported activities can improve knowledge acquisition and learner confidence while encouraging explanation-seeking and debugging-oriented behaviors (Xu et al., 2026). Together, these findings support a process-aware approach to programming assessment in which AI is embedded within explicit scaffolding, reflective prompting, and guided verification practices rather than treated solely as a productivity tool.

Taken together, the reviewed literature yields three central conclusions. First, ACA systems remain indispensable for scalable programming assessment, yet dominant im-

plementations continue to focus primarily on output-level evaluation and provide limited visibility into learners' reasoning processes and AI reliance (Becker et al., 2023; Combéfis, 2022; Messer et al., 2024). Second, generative AI can enhance programming learning, but its educational value is strongly mediated by learner characteristics, instructional design, and students' verification practices (Güner et al., 2025; Prather et al., 2024; Suh et al., 2025). Third, reflection-oriented interventions appear particularly effective for transforming tool-assisted task completion into meaningful learning, especially when reflection is timely, structured, and explicitly connected to revision decisions (Choi et al., 2023; Lee et al., 2025; Xie et al., 2023; Xu et al., 2026).

This thesis is situated at the intersection of these findings. Rather than proposing a new grading system, language model, or detection-only policy, it augments an automated code-assessment workflow with a mandatory post-submission reflection module. The design is grounded in the premise that AI-assisted programming may produce functionally correct solutions without ensuring conceptual understanding. Consequently, assessment systems should capture not only submitted outputs, but also how learners interpret feedback, revise solutions, and justify their own contributions. The primary contribution of this thesis is therefore not the replacement of ACA with AI, but the integration of correctness-based evaluation and metacognitive prompting within a unified operational workflow, evaluated through a control–experimental comparison centered on self-directed learning behavior and passive AI reliance.

Concept

This chapter presents the conceptual solution proposed in this thesis for addressing passive reliance on AI tools in programming education. The problem is not the existence of AI tools themselves, but the fact that conventional automated code assessment systems primarily evaluate output correctness while leaving the learning process largely invisible. Under these conditions, students may submit functionally correct code without demonstrating independent reasoning, and instructors receive limited evidence for distinguishing genuine understanding from external dependency. The concept developed here addresses this gap by extending ACA with a structured reflection layer that captures process-oriented learning evidence at the point of submission.

The proposed design can be summarized as an ACA-Reflection architecture. The established workflow of assignment distribution, submission, automated testing, and technical feedback is retained, but complemented by a mandatory reflective step immediately after submission. This timing is intentional: reflection is expected to be most reliable when learners still remember their strategy, difficulties, and use of support tools. Reflection is therefore treated as a native workflow state rather than a detached end-of-course survey. The design objective is twofold: to support students in more self-directed and critical AI use, and to provide instructors with analyzable behavioural evidence beyond final scores.

3.1 Conceptual Workflow and Data Logic

At system level, the concept forms a continuous pipeline with three connected layers. The first is the submission and grading core, where student code is uploaded and evaluated automatically. The second is a reflection capture layer, where structured and

open-ended prompts collect information about strategy, difficulties, AI usage, and revision intent. The third is an instructional review layer, where technical outcomes and reflection records are interpreted together. The key value is continuity: grading and reflection are not separate processes, but linked states of one submission lifecycle.

In a typical interaction, the student selects a task and submits code. The runner executes predefined tests and returns machine-readable output such as status, score, and feedback. Immediately after upload confirmation, the reflection interface is shown and requires completion before continuation. The system then stores technical and reflective records under shared identifiers (`userId`, `assignmentId`, `submissionId`). Teachers can inspect both dimensions in one view and optionally mark coherence to support data-quality filtering. This creates a traceable chain from attempt, to result, to learner explanation.

A central design decision is attempt-aware prompting. First-attempt prompts focus on learning and encountered difficulties, while second-attempt prompts focus on revision quality, improved understanding, and remaining uncertainty. This distinction follows the assumption that reflection should match cognitive phase. Revision questions before revision exists are weak indicators, while first-attempt diagnostics after revision miss adaptation quality. Attempt-aware structure therefore increases interpretability across submission cycles.

3.2 Positioning Against Alternatives

The concept defines improvement relative to baseline ACA. In baseline systems, evidence is mainly result-centric: pass/fail status, score, and attempt count. In the proposed architecture, evidence becomes result-plus-process. This does not guarantee better outcomes automatically, but it substantially improves pedagogical interpretation and supports targeted intervention. Two students with identical final scores can differ in reflective indicators of ownership, uncertainty, and strategy quality; the proposed model makes these differences visible.

Alternative approaches were considered. AI-generated code detection can support integrity enforcement, but it does not directly foster reflective self-regulation and may produce adversarial behaviour. Optional reflection outside the grading flow lowers friction but often reduces response quality and participation. AI-generated feedback without learner reflection may improve feedback richness but leaves ownership and conceptual depth ambiguous. Strict AI prohibition is difficult to enforce and poorly aligned with current software practice. Compared with these alternatives, the present

concept emphasizes accountable and pedagogically integrated AI use within normal workflow constraints.

3.3 Expected Value, Trade-offs, and Requirements

The concept provides several expected benefits: improved visibility of learner process, repeatable evidence structures for instructors and researchers, stronger metacognitive prompting at meaningful points in the workflow, and modular deployability in existing ACA infrastructures. At the same time, trade-offs are acknowledged. Mandatory reflection increases effort and may produce low-quality responses if prompt design is weak. Coherence checks add instructor workload. Self-report remains imperfect and can be biased. The concept mitigates these risks through structured fields, attempt-aware logic, and triangulation with objective submission data.

To transition from concept to implementation, the following requirements are derived: one reflection per submission, strict linkage to submission and assignment identifiers, role-based access control, attempt-sensitive prompt switching, required-field validation, and teacher-side coherence assessment. Non-functional requirements include low-latency interaction, robust linkage integrity, and clear UI communication. These requirements define the implementation boundary for the next chapter.

In sum, the conceptual contribution of this thesis is a workflow-integrated reflective ACA extension that makes learning processes observable and analyzable in AI-supported programming contexts. Its novelty lies less in creating a new grader and more in coupling automated correctness evaluation with structured metacognitive capture at the moment of decision-making. Relative to detection-only, optional-survey, or prohibition-focused alternatives, the concept offers a balanced and practical path toward more interpretable and pedagogically meaningful assessment.

Implementation

This chapter documents the implementation of the prototype designed in the previous chapter. The implementation objective is to operationalize the core concept of this thesis: extending an Automated Code Assessment (ACA) system with a mandatory reflection mechanism that captures process-oriented learning evidence in AI-supported programming tasks. The implementation does not attempt to build a full learning management system; instead, it focuses on the subset of features required to answer the research question and support future empirical evaluation.

The implemented prototype is built around three priorities. First, the conventional ACA workflow must remain stable, scalable, and asynchronous. Second, reflection must be integrated at the submission moment, not as a detached survey step. Third, data generated by students and the system must be traceable and analyzable in a mixed-method framework. These priorities guide architectural choices, API design, interface behavior, and persistence structure throughout the implementation.

4.1 System Architecture and Component Responsibilities

The prototype is implemented as a three-tier architecture with clear separation of concerns. The frontend is implemented in React with Vite and provides role-specific user interfaces for students and teachers. The backend is implemented in Node.js/Express and acts as orchestration and policy layer, including authentication, data access, and business rules. A separate Python runner service executes assignment test suites and returns grading outcomes asynchronously.

This separation is essential for two reasons. First, test execution may take longer than normal web interaction cycles, so the grading workload should not block the primary application server. Second, reflection interaction should start immediately after upload while grading can continue independently. The resulting architecture therefore combines synchronous user feedback with asynchronous evaluation.

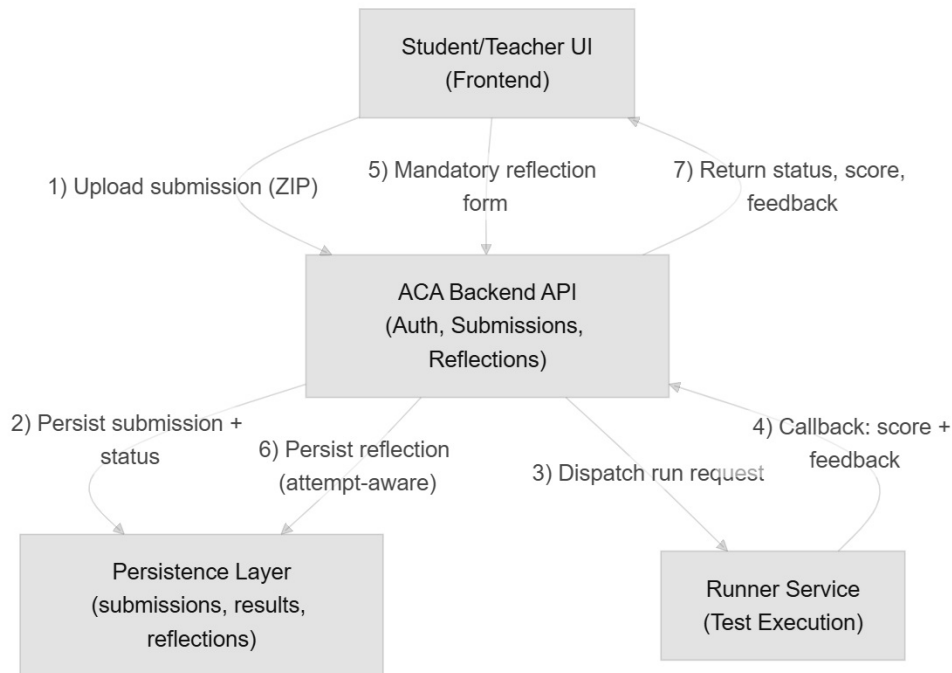


Figure 4.1: System architecture of the ACA-Reflection prototype, showing frontend, backend, runner, and data storage, including request/callback directions.

Communication between services uses HTTP with JSON payloads. Authentication is token-based, and role permissions are enforced by middleware in backend routes. Student-visible flows and teacher-visible flows share core data structures but differ in access scope and available actions.

4.2 Submission Lifecycle and Asynchronous Grading

A submission begins when a student selects an assignment and uploads a solution. The backend validates input (assignment identifier, file type, and attempt limits), stores the archive with a generated unique filename, creates a submission record, and dispatches metadata to the runner service. The runner extracts the archive, executes assignment-specific tests, and sends a callback to the backend with status, score, and feedback details.

The callback-based design avoids long synchronous waits and prevents frontend freez-

ing. It also improves failure isolation: if a runner job fails, the application remains responsive, and failure can be represented as a controlled submission state transition. This lifecycle is implemented through explicit status values (e.g., queued, processing, completed, failed), allowing both transparency in user interfaces and reproducible audit trails for analysis.

```

1 app.post('/api/submissions', authRequired, upload.single('file'), async (
    ↪ req, res) => {
2   const { assignmentId } = req.body;
3   const userId = req.user.id;
4   validateZip(req.file);
5   await ensureAttemptLimit(userId, assignmentId, 2);
6   const submission = await createSubmission({
7     userId,
8     assignmentId,
9     filename: req.file.filename,
10    status: 'queued'
11  });
12  await dispatchToRunner({
13    submissionId: submission.id,
14    assignmentId,
15    filename: submission.filename
16  });
17  await updateSubmissionStatus(submission.id, 'processing');
18  res.json({ submissionId: submission.id });
19 });

```

Listing 4.1: Demo implementation of a submission route (Node.js/Express)

This code shows the integration point where technical grading and reflection flow diverge: once `submissionId` exists, the frontend can immediately open reflection while runner evaluation proceeds asynchronously.

4.3 Reflection Module: Workflow-Coupled Design

The reflection module is the core implementation contribution of this thesis. Its defining characteristic is workflow coupling: reflection is triggered directly after successful submission, not delayed to a separate page or optional questionnaire. This design addresses a common data-quality issue in educational systems, where optional post-task surveys often yield incomplete and biased responses. The implementation distinguishes first and second attempts per assignment.

The attempt type is computed from prior submissions of the same user and assignment

and then used to branch reflection processing. For first attempts, required prompts focus on learning and encountered difficulty; for second attempts, required prompts focus on revision decisions, newly gained understanding, and remaining uncertainties.

```

1 function getSubmissionAttemptNumber(submission) {
2   const siblings = database.submissions.filter(
3     (s) => s.userId === submission.userId && s.assignmentId === submission.
      ↪ assignmentId
4   );
5   siblings.sort((a, b) => {
6     const ta = new Date(a.createdAt).getTime();
7     const tb = new Date(b.createdAt).getTime();
8     if (ta !== tb) return ta - tb;
9     return a.id - b.id;
10  });
11  const idx = siblings.findIndex((s) => s.id === submission.id);
12  return idx >= 0 ? idx + 1 : 1;
13 }

```

Listing 4.2: Submission attempt number calculation per user and assignment

```

1 const attemptNumber = getSubmissionAttemptNumber(submission);
2 const isSecondAttempt = attemptNumber >= 2;
3 if (isSecondAttempt) {
4   const r1 = String(revisionChangeText || '').trim();
5   const r2 = String(revisionUnderstandText || '').trim();
6   const r3 = String(revisionNextIterationText || '').trim();
7   if (!r1 || !r2 || !r3) {
8     return res.status(400).json({
9       error: 'Second-attempt reflection requires all three revision prompts
      ↪ (change after feedback, understanding, next iteration).'
10    });
11  }
12 } else {
13   const req1 = String(learnedText || '').trim();
14   const req2 = String(difficultiesText || '').trim();
15   if (!req1 || !req2 || wroteCodeMyself == null || wroteCodeMyself === ''
      ↪ ||
16     !String(aiToolUsage || '').trim() || reflectedOnApproach == null ||
      ↪ reflectedOnApproach === '') {
17     return res.status(400).json({ error: 'All first-attempt reflection
      ↪ fields are required.' });
18   }
19 }

```

Listing 4.3: Backend attempt-specific reflection validation (first vs second attempt)

This attempt-aware validation is enforced in both layers: the frontend blocks submission when required fields are missing, while the backend re-validates payload consistency against the detected attempt type. This dual enforcement prevents malformed reflection records and improves cross-record comparability for subsequent analysis.

```

1 const validate = () => (isSecondAttempt ? validateAttempt2() :
    ↪ validateAttempt1());
2 const hasMissingRequired = validate().length > 0;
3 const handleSubmit = async (e) => {
4   e.preventDefault();
5   const missing = validate();
6   if (missing.length > 0) {
7     alert('Please answer all required questions:\n ${missing.map((m) => m.
        ↪ label).join('\n')});
8     return;
9   }
10
11 };

```

Listing 4.4: Frontend required-field enforcement in the reflection modal

4.4 Reflection Data Schema and Validation Logic

Reflection records combine structured and open-ended fields. Structured fields (Likert/categorical/slider) support quantitative comparability, while open-ended fields capture explanatory detail not representable on fixed scales.

```

1 const submission = database.submissions.find(s => s.id === parseInt(
    ↪ submissionId));
2 if (!submission) {
3   return res.status(404).json({ error: 'Submission not found' });
4 }
5 if (submission.userId !== req.user.id) {
6   return res.status(403).json({ error: 'You can only add reflections to
    ↪ your own submissions' });
7 }
8 const existing = database.reflections?.find(r => r.submissionId ===
    ↪ parseInt(submissionId));
9 if (existing) {
10   return res.status(400).json({ error: 'Reflection already submitted for
    ↪ this submission' });
11 }

```

Listing 4.5: Ownership and duplicate prevention in reflection creation route

Each reflection is linked to a concrete `submissionId`, `assignmentId`, and `userId`, and stores `reflectionAttempt`. To preserve interpretability against grading outcomes, backend route logic enforces ownership consistency and uniqueness at submission level (Listing 4.5). This ensures that each submission can contribute at most one valid reflection record from its owner.

4.5 Teacher Review and Coherence Assessment

Teacher functionality is implemented as a dedicated review interface where submission details, test outcomes, and full reflection responses are shown in one view. A teacher can assign a coherence label (e.g., logical/not logical) and optional notes. This mechanism acts as a lightweight quality-control layer before analysis; it is not intended as full qualitative coding.

From a technical perspective, this required role-protected update routes and additional reflection fields (`logicalCheck`, `teacherNotes`, `assessedBy`, `assessedAt`). From a methodological perspective, it provides a transparent criterion to flag responses that may be empty, inconsistent, or likely non-authentic.

4.6 Data Model and Traceability for Analysis

The persistence model includes five core entity groups: users, assignments, submissions, results, and reflections. The submission record is the linking anchor between technical and reflective dimensions. This model supports time-based and attempt-based analysis while preserving role boundaries in access.

Traceability is a primary implementation requirement. A reflection without submission context has low analytical value; a score without process context has limited pedagogical meaning. By explicitly connecting both through identifiers and timestamps, the prototype enables analyses such as first-attempt correctness vs reflection depth, or revision-focused responses vs score change between attempts.

4.7 UI Implementation Decisions and Usability Trade-offs

The student interface prioritizes clarity and low friction during submission and reflection. Validation errors are displayed in plain language, and required inputs are high-



Figure 4.2: Entity relationship view of users, assignments, submissions, results, and reflections, including key linkage fields.

lighted before save. The teacher interface prioritizes overview and drill-down review efficiency through tables and detail modals.

A deliberate trade-off is the mandatory nature of reflection. This increases response completeness but can increase user friction if prompts are poorly designed. The implementation addresses this by keeping prompt wording concise, attempt-aware, and directly connected to recent actions. Another trade-off concerns rubric complexity in teacher assessment: a binary coherence marker was chosen for operational simplicity, though richer rubrics could be added in future versions.

4.8 Difficult and Innovative Aspects of the Prototype

The most difficult part of implementation was coordinating asynchronous runner execution with immediate post-submission reflection interaction while maintaining clean data linkage. This required careful state design and strict identifier propagation from upload to reflection save.

A second difficult aspect was attempt-aware schema switching. Frontend and backend had to remain consistent in prompt logic and field validation, otherwise data reliability would degrade. A third challenge was ensuring role-safe data operations while preserving a convenient workflow for teacher review.

The innovative aspect of the prototype is not a new grading algorithm. Its novelty lies in embedding metacognitive capture directly into a functioning ACA pipeline, producing unified output-plus-process evidence. In practical terms, the system upgrades ACA from “does the code pass?” to “does the code pass, and what did the learner report about strategy, AI use, and revision?”

4.9 Implementation Outcome and Readiness

The implemented prototype fulfills its intended scope. Students can submit assignments, receive automated grading outcomes, and complete mandatory attempt-specific reflection forms. Teachers can review integrated records and annotate reflection coherence. The system stores analyzable linked data across technical and reflective dimensions, and preserves sufficient traceability for mixed-method evaluation workflows.

Although large-scale participant testing is outside the current implementation chapter, the prototype establishes a technically robust foundation for future empirical studies. It demonstrates that reflection-oriented process capture can be integrated into existing ACA architecture with moderate complexity and clear pedagogical relevance, which is the core implementation contribution of this thesis.

Evaluation

This chapter evaluates the implemented prototype against the central problem of this thesis: standard automated code assessment captures correctness outcomes but provides limited visibility into learning process, revision logic, and AI-use behaviour. The evaluation combines a technical validation of the system with an exploratory user study to assess both implementation correctness and its potential impact on student behaviour.

The purpose is therefore twofold: first, to determine whether the implementation is technically correct, whether collected data is analyzable, and whether the system is ready for empirical deployment; and second, to obtain initial qualitative insights into how the reflection-enhanced workflow influences learners' interaction with programming tasks and AI tools. The evaluation follows a structured scenario-based approach. Scenarios were designed to cover normal and edge-case behaviour across the full workflow: successful submission with complete reflection, submission with incomplete reflection, second attempt with revision-focused reflection, attempt-limit checks, runner callback updates, and teacher coherence assessment. For each scenario, expected system behaviour was defined before execution, then compared with observed behaviour in UI and persisted records. This method allows reproducible validation of the implementation even without participant sampling.

The core submission-to-reflection path was validated first. After assignment selection and ZIP upload, the backend acknowledged the submission, created a submission record, and dispatched runner execution. Immediately after acknowledgment, the reflection modal opened and required completion of mandatory fields. Reflection records were then stored with the correct linkage identifiers (submissionId, assignmentId, userId) and attempt marker. Runner callback updates were reflected in result

records and visible in the student interface. The end-to-end chain matched design expectations.

The figure consists of two screenshots of a web application interface for student submissions. Both screenshots show a sidebar with assignment cards for 'FizzBuzz', 'CSV Statistics', and 'Vector2D'. The main content area displays a 'Reflection (required)' form for the 'FizzBuzz' assignment. The form includes several fields: 'What did you learn from this exercise?', 'What difficulties did you encounter while solving the task?', 'I wrote the code mostly myself (1-5)', 'AI tool usage during this task', 'I understood AI-generated code before submit (1-5)', 'I reflected on my problem-solving strategy (1-5)', and a slider for 'Own work vs external help: 50% own work'. A 'Submit reflection' button is located at the bottom right of the form. In the top screenshot, the button is inactive (greyed out). In the bottom screenshot, the button is active (blue with white text). The background of the interface shows a submission status of 'Submission received. We will notify you once grading is complete.' and a 'Submit assignment' button at the bottom.

Figure 5.1: Reflection form completion gating in the student submission flow. With incomplete inputs (top), the Submit reflection action remains inactive; after all required open-ended and structured fields are completed (bottom), the button becomes active and reflection submission is enabled.

Validation robustness was tested by intentionally providing incomplete reflection data. In the frontend, final assignment submission remained disabled until all required reflection fields were completed, preventing incomplete payloads from being sent. At the API level, manually crafted incomplete or malformed requests were still rejected by backend validation. This dual-layer control confirms that completeness enforcement is functional and that stored records satisfy the schema requirements for later analysis.

Attempt-dependent logic was tested by submitting a second attempt for the same assignment. The system correctly switched from first-attempt prompts to revision-focused prompts and stored attempt-specific fields under a new submission linked to the same

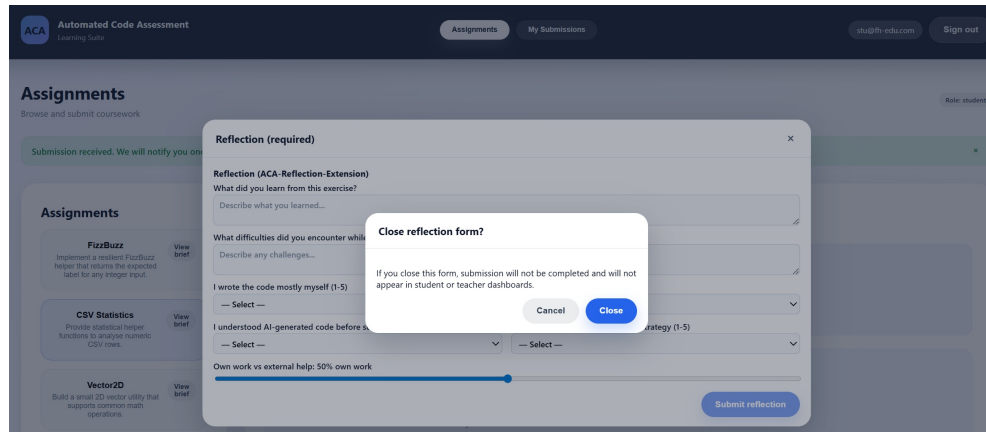


Figure 5.2: Confirmation dialog shown when a student attempts to close the mandatory reflection form before completion. The warning makes clear that submission will not be finalized and no record will appear in student or teacher dashboards unless the reflection process is completed.

assignment and user. This behaviour is critical for interpretation of learning progression, because first-attempt reflection and revision reflection represent different cognitive phases and should not be mixed.

Teacher-side review was evaluated in dedicated scenarios. Submission detail pages displayed technical outcome and full reflection in one view, and coherence marking could be saved with optional notes. Role-protection rules correctly restricted assessment updates to teacher accounts. This verifies that the planned quality-control checkpoint is operational and traceable.

After the first submission, the reflection form captures initial learning outcomes, perceived difficulties, and self-reported use of external support. However, these first-attempt items do not yet provide direct evidence of revision behaviour after feedback. For this reason, the prototype introduces a second-attempt prompt set with a distinct pedagogical and analytical purpose. The second-attempt prompts are designed to operationalize reflective revision by requiring students to document concrete changes to their solution, explain why those changes should improve correctness or quality, and differentiate between newly gained understanding and remaining uncertainty. In contrast to first-attempt reflection, which is primarily diagnostic, second-attempt reflection is process-comparative: it captures whether feedback was interpreted and transformed into meaningful code-level adaptation.

From a methodological perspective, this distinction is important for evaluating self-directed learning in iterative programming tasks. A higher second-attempt score alone cannot explain how improvement occurred; students may improve through trial-and-

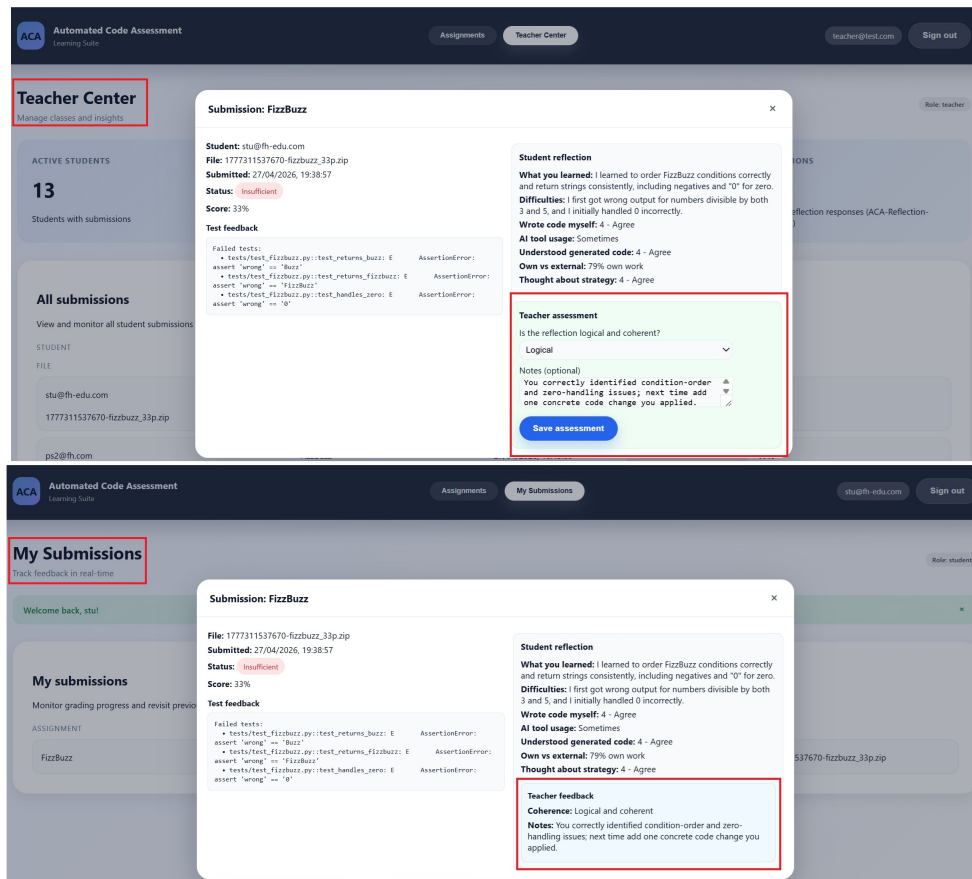


Figure 5.3: End-to-end reflection feedback loop across teacher and student views. In the Teacher Center (top), instructors review student reflections and record structured assessment comments; in My Submissions (bottom), students see the same evaluation outcome and feedback, ensuring transparent reflection-based guidance after grading.

error, external copying, or deliberate conceptual revision. The second-attempt prompts reduce this ambiguity by collecting explicit self-explanations of revision intent and reasoning. The form therefore supports interpretation of performance changes between attempts and provides qualitative evidence that complements technical outcome metrics. In the context of this thesis, the second-attempt reflection module serves as a bridge between automated feedback and learner agency: it makes revision decisions visible, analyzable, and comparable across submissions.

To summarize execution outcomes, Table 5.1 reports scenario-level expected behaviour. All defined scenarios were executed, and the observed system behaviour was consistent with the expected behaviour in all cases. Beyond workflow correctness, the evaluation examined data readiness for mixed-method analysis. The prototype now captures timestamped behavioural metrics and structured/open reflection variables in one linked model. This includes attempt counts, submission timestamps, test outcomes, AI-use

Assignments
Browse and submit coursework

Submission received. We will notify you once your submission is graded.

Assignments

- FizzBuzz**
Implement a resilient FizzBuzz helper that returns the expected label for any integer input. [View brief](#)
- CSV Statistics**
Provide statistical helper functions to analyse numeric CSV rows. [View brief](#)
- Vector2D**
Build a small 2D vector utility that supports common math operations. [View brief](#)

Reflection (required)

Reflection — second submission
This is your second attempt for this assignment. Answer all questions. They focus on what you changed after feedback and how your understanding evolved.

What did you change after feedback and reflection, and why should that improve your solution?
Describe concrete changes (logic, tests, structure, error handling) and why they should help.

Compared to your first submission, what do you understand better now about your own code or the problem—and what is still unclear?
Be specific about understanding vs remaining gaps.

If you had one more iteration, what would you still change and why?
Optional improvements you did not have time to make.

[Submit reflection](#)

Figure 5.4: Second-attempt reflection form capturing post-feedback changes, improvement rationale, and remaining uncertainties.

self-report, contribution indicators, open reflection text, and teacher coherence labels. These fields are sufficient to compute the indicators defined in the methodology and to perform qualitative coding of reflection content in future studies. Table 5.2 maps stored fields to analytical indicators.

Interpretation of results must distinguish what has been proven from what remains open. The evaluation shows that the implementation successfully solves the instrumentation problem: it captures technical and reflective evidence in one coherent workflow with reliable linkage.

With this boundary stated clearly, hypothesis status can be interpreted as follows. The implementation hypothesis is accepted: the ACA system can be extended with mandatory, attempt-aware reflection and teacher coherence review without breaking grading workflow.

The prototype is now technically stable, internally consistent, and analytically prepared for empirical study. In practical terms, it transforms ACA from output-only logging to output-plus-process evidence capture, which is the core contribution of this thesis implementation.

In conclusion, the evaluation demonstrates that the prototype has achieved deployment readiness for future classroom studies. The next research step is controlled empirical use with real participants to test whether the observed instrumentation capabilities translate into measurable changes in self-directed learning behaviour and passive AI reliance.

Table 5.1: Scenario-based evaluation overview

ID	Scenario	Expected Behaviour
S1	First valid ZIP submission	Upload is accepted; submissionId is created; status transitions to processing
S2	Post-upload reflection trigger	Reflection form opens immediately after upload acknowledgment
S3	Incomplete reflection submission	Submission is blocked until all required fields are completed
S4	Backend reflection validation	Invalid or incomplete API payloads are rejected
S5	Second attempt prompt logic	Revision prompts are displayed; reflectionAttempt is set to 2
S6	Runner callback update	Status, score, test counts, and feedback are updated correctly
S7	Teacher coherence update	Teacher can save coherence label and notes
S8	Teacher-endpoint role protection	Unauthorized student requests to teacher-only endpoints are rejected

5.1 Exploratory User Study with Target Groups

In addition to the technical evaluation of the implemented prototype, an exploratory user study was conducted with two small target groups in order to gain preliminary insights into how the reflection-enhanced Automated Code Assessment (ACA) system influences student behaviour in practice. Table 5.3 presents the participant profile in the control and experimental groups, including age, educational level, prior experience, and experience with ACA.

The control group used the baseline ACA workflow without mandatory reflection prompts. After task completion, participants answered a short survey on AI use and perceived measures to reduce over-reliance on AI tools (Table 5.4).

The experimental group used the reflection-enhanced ACA workflow. After each submission, participants completed structured and open-ended reflection prompts. Second-attempt prompts focused explicitly on revision decisions after feedback (Table 5.5). To capture perceived behavioural effects, participants in the experimental group also answered the following additional item: *“Do you think adding reflection questions after submission changes how you approach problem-solving in future programming tasks? If yes, how? If no, why not?”*

Given the small exploratory sample, findings are reported as traceable qualitative pat-

Table 5.2: Stored variables and derived indicators for later analysis

Data Source	Stored Fields	Derived Indicator / Use
submissions	createdAt, assignmentId, userId, status	Time to first submission, attempts per task, submission progression
results	score, passedTests, totalTests, feedback	First-attempt correctness, improvement between attempts, test-level outcome interpretation
reflections (structured)	AI-use frequency, own-vs-external contribution, reflection Likert items	Self-reported AI reliance, perceived ownership, reflection intensity
reflections (open text)	Learned text, difficulties text, revision-focused responses	Thematic analysis of learning strategies and revision reasoning
Teacher assessment fields	logicalCheck, teacherNotes, assessedAt	Data quality filtering and sensitivity checks

terns rather than inferential statistics. In the experimental group, three participants described reflection prompts as helpful for re-checking understanding and guiding revision, while two participants reported little or no clear reduction in AI reliance. One participant explicitly reported using AI to generate parts of the reflection responses, and one participant raised honesty/monitoring concerns when reporting AI usage. Across groups, response patterns differed in emphasis. Control-group responses were predominantly outcome-oriented (efficiency, task completion), whereas reflection-group responses more often referred to understanding, revision, and strategy awareness. These patterns suggest that integrated reflection may shift attention toward process-level engagement, while not eliminating AI use. For transparency and traceability, full item-level response tables are provided in Appendix A.

Despite these indications, interpretation remains limited by the small sample size and exploratory design. The results are therefore indicative rather than generalizable and should be treated as groundwork for larger, controlled follow-up studies.

Table 5.3: Participant Demographics

Group	Participant No.	Age	Study Level / Occupation
Control Group	1	35	Master's student in Software Engineering
	2	37	IT employee
	3	22	Student in Software Engineering
	4	40	IT employee
	5	45	Graduate in Software Engineering
Experimental Group	1	29	Graduate in Software Engineering
	2	34	Student in Software Engineering
	3	27	Student in Software Engineering
	4	29	Student in Software Engineering
	5	38	PhD student in IT

Table 5.4: Survey questions for the control group

Question No.	Survey Question	Response Options
1	Did you use any AI tool (e.g., ChatGPT) for this task?	Yes / No
2	(If yes) How often did you use AI during this task?	1 = Rarely; 2 = Sometimes; 3 = Often; 4 = Very often
3	Approximately what percentage of your final solution was directly based on AI-generated code?	0–100%
4	What measures do you think could help reduce over-reliance on AI tools (e.g., ChatGPT) in programming exercises?	Open-ended response

Table 5.5: Reflection prompts used in the experimental group by submission attempt

Attempt	Questions
Attempt 1 (First Submission)	1. What did you learn from this exercise? 2. What difficulties did you encounter while solving the task? 3. I wrote the code mostly myself (1–5). 4. AI tool usage during this task. 5. I understood AI-generated code before submission (1–5). 6. I reflected on my problem-solving strategy (1–5). 7. Own work vs. external help (% own work).
Attempt 2 (Second Submission)	1. What did you change after feedback and reflection, and why should that improve your solution? 2. Compared to your first submission, what do you understand better now about your own code or the problem, and what is still unclear? 3. If you had one more iteration, what would you still change and why?

Conclusion and Outlook

This thesis addressed a central challenge in contemporary programming education: automated code assessment systems reliably evaluate correctness, but they provide limited visibility into the learning process, especially when students use AI support during problem solving. In this context, correctness alone is no longer sufficient to interpret learning quality. The work therefore focused on extending an existing ACA workflow with a mandatory, attempt-aware reflection module that captures process-oriented evidence alongside technical outcomes.

The main contribution is a working prototype that integrates submission handling, asynchronous grading, reflection capture, and teacher-side coherence review in one coherent system. The implementation demonstrates that reflective instrumentation can be embedded into standard ACA architecture without replacing core grading logic. The resulting data model links submission events, grading outcomes, and reflection responses through stable identifiers and timestamps, enabling traceable mixed-method analysis.

At the technical level, the prototype fulfills its intended scope. Submission-to-runner orchestration, callback-based result updates, required reflection validation, and role-based teacher assessment operate as designed. At the methodological level, the system now produces analyzable records that support both quantitative indicators (e.g., attempts, time-related metrics, score progression) and qualitative interpretation (reflection content, revision rationale, coherence annotations).

In this sense, the thesis achieves its implementation objective: it transforms an output-centric ACA pipeline into an output-plus-process evidence pipeline suitable for future empirical deployment. The contribution is practical and research-oriented at the same

time: it provides an operational artifact and a structured basis for evaluating learning behaviour in AI-supported settings.

6.1 Interpretation of Results

The results should be interpreted as implementation and evidence-readiness results. The prototype shows that the proposed concept is feasible, coherent, and technically stable within thesis scope. It also confirms that reflection can be collected at submission time in a way that remains linked to concrete technical outcomes and revision cycles.

The educational implications are promising, but claims must remain bounded. The current findings indicate that the system can make reflective behaviour visible and therefore support richer pedagogical interpretation than score-only views. However, measurable effect on student learning behaviour cannot be inferred from implementation scenarios alone. The thesis therefore contributes validated infrastructure for effect testing, not final proof of behavioural impact. Complementing the technical evaluation, exploratory user study results provide initial qualitative indications that the reflection-enhanced workflow may promote more process-oriented thinking and self-explanation. However, due to the small sample size and exploratory design, these observations remain indicative rather than conclusive and require validation through controlled empirical studies.

6.2 Limitations

This work demonstrates the technical validity of the implementation and its methodological readiness; however, the educational impact at the level of a broader student population remains an open question. Although a small-scale exploratory user study was conducted with 10 participants split into control and experimental groups, the sample size and experimental design are not sufficient for statistical testing of behavioural hypotheses. As a result, any claims regarding behavioural change cannot be statistically validated. The main evaluation of the system is scenario-based, which confirms the technical correctness of the implementation and the readiness of the collected data for further analysis. However, it does not replace longitudinal or controlled studies in real classroom environments. In addition, indicators related to AI usage are partially based on self-reported data, which may introduce bias, even though consistency checks were performed by the instructor.

6.3 Outlook and Future Work

The next step is empirical deployment in real programming courses. The implemented platform should be used with student cohorts across multiple assignments so that reflective-process indicators and grading outcomes can be analyzed over time. Depending on course constraints, future evaluation can use a pre-post or crossover design to test behavioural hypotheses under realistic educational conditions.

Future studies should combine system logs, structured reflection variables, and open-text coding to examine whether higher reflection quality is associated with better revision decisions and reduced passive AI reliance. Teacher coherence labels can be used as an initial quality filter, followed by deeper qualitative coding. This would allow direct testing of whether process-aware ACA instrumentation leads to measurable learning benefits beyond correctness metrics.

Several technical extensions can further strengthen the platform. Prompt sets can be adapted by assignment type and learner level. Reflection quality support can be expanded with lightweight NLP-assisted analysis while retaining teacher oversight. Additional analytics dashboards can help instructors detect patterns in revision behaviour and AI-use self-report at class level. Integration with code-diff metadata could also improve triangulation between reported and actual changes between attempts.

In conclusion, this thesis provides a practical and research-ready foundation for process-aware assessment in AI-supported programming education. The implemented ACA-Reflection prototype demonstrates that reflective evidence can be captured in a structured, traceable, and scalable way within existing grading workflows. While population-level educational impact remains to be tested, the system now enables that next stage of research with significantly improved methodological and technical readiness.

Appendix **A**

Survey Response Tables

A.1 Control Group: Full Item-Level Responses

A.2 Experimental Group: Full Item-Level Responses

Table A.1.1: Control Group Survey Responses (Participant 1)

Survey Question	Participant 1
Did you use any AI tool (e.g., ChatGPT) for this task?	Yes
<i>(Only if Yes)</i> How often did you use AI during this task?	2 (Sometimes)
Approximately what percentage of your final solution was directly based on AI-generated code?	25%
What measures do you think could help reduce over-reliance on AI tools (e.g., ChatGPT) in programming exercises?	Honestly, I think it depends on motivation. If the task is boring or repetitive, people will always use AI. Maybe making assignments more interesting or graded on explanation instead of just output would help. I do not think banning AI is realistic.
Result (%)	50

Table A.1.2: Control Group Survey Responses (Participant 2)

Survey Question	Participant 2
Did you use any AI tool (e.g., ChatGPT) for this task?	Yes
<i>(Only if Yes)</i> How often did you use AI during this task?	4 (Very often)
Approximately what percentage of your final solution was directly based on AI-generated code?	60%
What measures do you think could help reduce over-reliance on AI tools (e.g., ChatGPT) in programming exercises?	I think deadlines are part of the problem. When I am stressed, I just ask AI instead of thinking deeply. Maybe giving more time or breaking tasks into parts would help reduce that pressure.
Result (%)	90

Table A.1.3: Control Group Survey Responses (Participant 3)

Survey Question	Participant 3
Did you use any AI tool (e.g., ChatGPT) for this task?	Yes
<i>(Only if Yes)</i> How often did you use AI during this task?	3 (Often)
Approximately what percentage of your final solution was directly based on AI-generated code?	35%
What measures do you think could help reduce over-reliance on AI tools (e.g., ChatGPT) in programming exercises?	I do not think AI is the problem. Maybe assignments should require you to explain your process or defend your solution in class. That would stop blind copying.
Result (%)	100

Table A.1.4: Control Group Survey Responses (Participant 4)

Survey Question	Participant 4
Did you use any AI tool (e.g., ChatGPT) for this task?	Yes
<i>(Only if Yes)</i> How often did you use AI during this task?	3 (Often)
Approximately what percentage of your final solution was directly based on AI-generated code?	40–50%
What measures do you think could help reduce over-reliance on AI tools (e.g., ChatGPT) in programming exercises?	If I have to explain what I did, why I did it, and what the code is doing in my own words, I naturally rely less on AI because I cannot explain something I did not understand.
Result (%)	80

Table A.1.5: Control Group Survey Responses (Participant 5)

Survey Question	Participant 5
Did you use any AI tool (e.g., ChatGPT) for this task?	Yes
<i>(Only if Yes)</i> How often did you use AI during this task?	3 (Often)
Approximately what percentage of your final solution was directly based on AI-generated code?	50%
What measures do you think could help reduce over-reliance on AI tools (e.g., ChatGPT) in programming exercises?	Maybe having stricter rules during exercises would force more learning. But honestly, outside of class I will still use AI because it is faster. I think it depends on personal discipline too.
Result (%)	65

Table A.2.1: Experimental Group Responses (Participant 1)

Question	Participant 1
Attempt 1 (First Submission)	
What did you learn from this exercise?	I learned how FizzBuzz uses conditions to print different outputs depending on the number.
What difficulties did you encounter while solving the task?	Sometimes I was not sure if the mistake came from my logic or from implementation details.
I wrote the code mostly myself (1–5)	4
AI tool usage during this task	3
I understood AI-generated code before submission (1–5)	4
I reflected on my problem-solving strategy (1–5)	4
Own work vs external help (% own work)	75%
Result (%)	25
Attempt 2 (Second Submission)	
What did you change after feedback and reflection, and why should that improve your solution?	I removed unnecessary parts that made the code harder to read.
Compared to your first submission, what do you understand better now about your own code or the problem, and what is still unclear?	I understand better how my initial solution was more complicated than it needed to be. I still sometimes struggle with writing the most efficient version from the start.
If you had one more iteration, what would you still change and why?	I would try to plan the solution more before coding so that I do not need to fix as much afterward.
Result (%)	50
Additional Survey Item	
Do you think adding reflection questions after submission changes how you approach problem-solving in future programming tasks? If yes, how? If no, why not?	I actually liked the reflection questions after submitting the code. They made me go back and think about what I did instead of just moving on. But I also felt a bit stressed about estimating my AI usage because I was not always sure how accurate it is. I think this system might slightly reduce AI use, because it makes you more aware of what you did yourself, but I do not think it fully stops people from using AI.

Table A.2.2: Experimental Group Responses (Participant 2)

Question	Participant 2
Attempt 1 (First Submission)	
What did you learn from this exercise?	I learned how functions can return different values based on logical conditions.
What difficulties did you encounter while solving the task?	Choosing the best approach.
I wrote the code mostly myself (1–5)	3
AI tool usage during this task	4
I understood AI-generated code before submission (1–5)	4
I reflected on my problem-solving strategy (1–5)	3
Own work vs external help (% own work)	60%
Result (%)	80
Attempt 2 (Second Submission)	
What did you change after feedback and reflection, and why should that improve your solution?	I rewrote parts of the AI-generated solution to understand it better and reduce unnecessary complexity.
Compared to your first submission, what do you understand better now about your own code or the problem, and what is still unclear?	I understand the logic more clearly, but I still rely on AI for structure sometimes.
If you had one more iteration, what would you still change and why?	If I had another iteration, I would try solving the problem fully before using AI.
Result (%)	100
Additional Survey Item	
Do you think adding reflection questions after submission changes how you approach problem-solving in future programming tasks? If yes, how? If no, why not?	To be honest, I even used AI to answer some of the reflection questions. I was tired after finishing the code, so I just wanted to finish everything quickly. I do not think these questions really reduce AI usage. People who want to use AI will still use it, even for the reflection part.

Table A.2.3: Experimental Group Responses (Participant 3)

Question	Participant 3
Attempt 1 (First Submission)	
What did you learn from this exercise?	I learned this is a good exercise for practicing Python basics.
What difficulties did you encounter while solving the task?	Starting the task.
I wrote the code mostly myself (1–5)	2
AI tool usage during this task	5
I understood AI-generated code before submission (1–5)	3
I reflected on my problem-solving strategy (1–5)	2
Own work vs external help (% own work)	30%
Result (%)	70
Attempt 2 (Second Submission)	
What did you change after feedback and reflection, and why should that improve your solution?	I tried to rewrite the code myself instead of just following AI output.
Compared to your first submission, what do you understand better now about your own code or the problem, and what is still unclear?	I understand some parts better now, but I still cannot solve the problem without help.
If you had one more iteration, what would you still change and why?	I would prefer step-by-step guidance before doing the full task.
Result (%)	20
Additional Survey Item	
Do you think adding reflection questions after submission changes how you approach problem-solving in future programming tasks? If yes, how? If no, why not?	The reflection part was actually useful for me. When I had to explain what I learned, I realized there were parts of my code I did not fully understand. That made me go back and check it again. I think this can reduce AI usage indirectly, because you become more aware of your understanding and rely less on AI over time.

Table A.2.4: Experimental Group Responses (Participant 4)

Question	Participant 4
Attempt 1 (First Submission)	
What did you learn from this exercise?	I did not learn much.
What difficulties did you encounter while solving the task?	Understanding the problem.
I wrote the code mostly myself (1–5)	4
AI tool usage during this task	3
I understood AI-generated code before submission (1–5)	4
I reflected on my problem-solving strategy (1–5)	4
Own work vs external help (% own work)	70%
Result (%)	100
Attempt 2 (Second Submission)	
What did you change after feedback and reflection, and why should that improve your solution?	I added comments.
Compared to your first submission, what do you understand better now about your own code or the problem, and what is still unclear?	No difference.
If you had one more iteration, what would you still change and why?	I had nothing to change.
Result (%)	100
Additional Survey Item	
Do you think adding reflection questions after submission changes how you approach problem-solving in future programming tasks? If yes, how? If no, why not?	I do not really like having to report AI usage percentage. It makes me feel like I am being monitored, and I am not always honest about it. I do not think it reduces AI use much. If anything, it might make students underreport instead of actually using AI less.

Table A.2.5: Experimental Group Responses (Participant 5)

Question	Participant 5
Attempt 1 (First Submission)	
What did you learn from this exercise?	I learned how to complete the task faster using AI assistance.
What difficulties did you encounter while solving the task?	Understanding some parts of AI-generated code.
I wrote the code mostly myself (1–5)	3
AI tool usage during this task	4
I understood AI-generated code before submission (1–5)	3
I reflected on my problem-solving strategy (1–5)	3
Own work vs external help (% own work)	50%
Result (%)	100
Attempt 2 (Second Submission)	
What did you change after feedback and reflection, and why should that improve your solution?	I removed unnecessary AI-generated complexity.
Compared to your first submission, what do you understand better now about your own code or the problem, and what is still unclear?	I understand the code better, but I still rely on AI.
If you had one more iteration, what would you still change and why?	I would solve the solution first before using AI.
Result (%)	100
Additional Survey Item	
Do you think adding reflection questions after submission changes how you approach problem-solving in future programming tasks? If yes, how? If no, why not?	The reflection questions were okay, but I only really started thinking properly after I had to answer them. Before that I was just focused on finishing the task as fast as possible. Maybe it can reduce AI use a little bit, because it forces you to look back at your work, but I think habits matter more than the questions themselves.

Acronyms

ACA	Automated Code Assessment
AI	Artificial Intelligence
API	Application Programming Interface
CSV	Comma-Separated Values
DOI	Digital Object Identifier
ER	Entity Relationship
FK	Foreign Key
HTTP	Hypertext Transfer Protocol
JWT	JSON Web Token
LLM	Large Language Model
LMS	Learning Management System
NLP	Natural Language Processing
PK	Primary Key
REST	Representational State Transfer
RQ	Research Question
SPA	Single-Page Application
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
UUID	Universally Unique Identifier

Bibliography

- Akçapınar, G. & E. Sidan (2024). “AI chatbots in programming education: guiding success or encouraging plagiarism”. In: *Discover Artificial Intelligence*. DOI: [10.1007/s44163-024-00203-7](https://doi.org/10.1007/s44163-024-00203-7) (cit. on p. 7).
- Albadarin, Y., M. Saqr, N. Pope, et al. (2024). “A systematic literature review of empirical research on ChatGPT in education”. In: *Discover Education*. DOI: [10.1007/s44217-024-00138-2](https://doi.org/10.1007/s44217-024-00138-2) (cit. on p. 7).
- Azaiz, I., N. Kiesler & S. Strickroth (2024). “Feedback-generation for programming exercises with GPT-4”. In: *Proc. ITiCSE*. DOI: [10.1145/3649217.3653594](https://doi.org/10.1145/3649217.3653594) (cit. on p. 7).
- Becker, B. A., P. Denny, J. Finnie-Ansley, et al. (2023). “Programming is hard—or at least it used to be”. In: *Proc. 54th ACM Technical Symposium on Computer Science Education (SIGCSE)*. DOI: [10.1145/3545945.3569759](https://doi.org/10.1145/3545945.3569759) (cit. on pp. 8, 10).
- Choi, H., J. Jovanovic, O. Poquet, et al. (2023). “The benefit of reflection prompts for encouraging learning with hints in an online programming course”. In: *The Internet and Higher Education* 58. DOI: [10.1016/j.iheduc.2023.100903](https://doi.org/10.1016/j.iheduc.2023.100903) (cit. on pp. 8, 10).
- Combéfis, S. (2022). “Automated code assessment for education: Review, classification and perspectives on techniques and tools”. In: *Software* 1.1. DOI: [10.3390/software1010002](https://doi.org/10.3390/software1010002) (cit. on pp. 8, 10).
- Fan, G., D. Liu, R. Zhang, et al. (2025). “The impact of AI-assisted pair programming on student motivation, programming anxiety, collaborative learning, and programming performance”. In: *International Journal of STEM Education*. DOI: [10.1186/s40594-025-00537-3](https://doi.org/10.1186/s40594-025-00537-3) (cit. on p. 8).
- Gambo, O., O. J. Onifade, W. Olanrewaju-Smart, et al. (2026). “Impact of ChatGPT on learning outcomes and performance of students in computer programming courses:

- a mixed-method approach”. In: *Discover Education*. DOI: [10.1007/s44217-026-01142-4](https://doi.org/10.1007/s44217-026-01142-4) (cit. on p. 8).
- Garcia, M. B. (2025). “Teaching and learning computer programming using ChatGPT: A rapid review of literature amid the rise of generative AI technologies”. In: *Education and Information Technologies*. DOI: [10.1007/s10639-025-13452-5](https://doi.org/10.1007/s10639-025-13452-5) (cit. on p. 8).
- Güner, H. & E. Er (2025). “AI in the classroom: Exploring students’ interaction with ChatGPT in programming learning”. In: *Education and Information Technologies*. DOI: [10.1007/s10639-025-13337-7](https://doi.org/10.1007/s10639-025-13337-7) (cit. on pp. 8, 10).
- Hooq, M., Y. Shi, J. Leinonen, et al. (2024). “Detecting ChatGPT-generated code submissions in a CS1 course using machine learning models”. In: *Proc. 55th ACM Technical Symposium on Computer Science Education (SIGCSE)*. DOI: [10.1145/3626252.3630826](https://doi.org/10.1145/3626252.3630826) (cit. on p. 9).
- Joshi, I., R. Budhiraja, H. Dev, et al. (2024). “ChatGPT in the classroom: An analysis of its strengths and weaknesses for solving undergraduate computer science questions”. In: *Proc. 55th ACM Technical Symposium on Computer Science Education (SIGCSE)*. DOI: [10.1145/3626252.3630803](https://doi.org/10.1145/3626252.3630803) (cit. on p. 9).
- Jukiewicz, M. (2024). “The future of grading programming assignments in education: The role of ChatGPT in automating the assessment and feedback process”. In: *Thinking Skills and Creativity*. DOI: [10.1016/j.tsc.2024.101522](https://doi.org/10.1016/j.tsc.2024.101522) (cit. on pp. 7, 9).
- Lee, H.-Y., P.-H. Chen, C.-J. Lin, et al. (2025). “Leveraging ChatGPT for personalized reflective learning in programming education: effects on self-efficacy, higher-order thinking, and project implementation skills”. In: *Education and Information Technologies*. DOI: [10.1007/s10639-025-13733-z](https://doi.org/10.1007/s10639-025-13733-z) (cit. on pp. 9, 10).
- Messer, M., N. C. C. Brown, M. Kölling & M. Shi (2024). “Automated grading and feedback tools for programming education: A systematic review”. In: *ACM Transactions on Computing Education*. DOI: [10.1145/3636515](https://doi.org/10.1145/3636515) (cit. on pp. 7, 10).
- Phung, T., V.-A. Padurean, J. Cambrono, et al. (2023). “Generative AI for programming education: Benchmarking ChatGPT, GPT-4, and human tutors”. In: *Proc. ACM Conference on International Computing Education Research (ICER)*. DOI: [10.1145/3568812.3603476](https://doi.org/10.1145/3568812.3603476) (cit. on p. 9).
- Prather, J., B. N. Reeves, J. Leinonen, et al. (2024). “The widening gap: The benefits and harms of generative AI for novice programmers”. In: *Proc. ACM Conference on International Computing Education Research (ICER)*. DOI: [10.1145/3632620.3671116](https://doi.org/10.1145/3632620.3671116) (cit. on pp. 9, 10).

- Stoyanova, D., S. Stoyanova-Petrova, S. Shotarova, et al. (2025). “ChatGPT in programming education: An empirical study on its impact on student performance, creativity, and teamwork”. In: *Education Sciences*. DOI: [10.3390/educsci16010019](https://doi.org/10.3390/educsci16010019) (cit. on p. 9).
- Suh, J., K. Lee & J. Lee (2025). “Programming education with ChatGPT: outcomes for beginners and intermediate students”. In: *Education and Information Technologies*. DOI: [10.1007/s10639-025-13542-4](https://doi.org/10.1007/s10639-025-13542-4) (cit. on pp. 9, 10).
- Sun, D., A. Boudouaia, C. Zhu & Y. Li (2024). “Would ChatGPT-facilitated programming mode impact college students’ programming behaviors, performances, and perceptions? An empirical study”. In: *International Journal of Educational Technology in Higher Education*. DOI: [10.1186/s41239-024-00446-5](https://doi.org/10.1186/s41239-024-00446-5) (cit. on p. 9).
- Wang, H., C. Wang, Z. Chen, et al. (2025). “Impact of AI-agent-supported collaborative learning on the learning outcomes of university programming courses”. In: *Education and Information Technologies*. DOI: [10.1007/s10639-025-13487-8](https://doi.org/10.1007/s10639-025-13487-8) (cit. on p. 9).
- Xie, B., J. O. Lim, P. K. D. Pham, et al. (2023). “Developing novice programmers’ self-regulation skills with code replays”. In: *Proc. ACM Conference on International Computing Education Research (ICER)*. DOI: [10.1145/3568813.3600127](https://doi.org/10.1145/3568813.3600127) (cit. on pp. 9, 10).
- Xu, J., D. Sun, Y. Feng, et al. (2026). “Empirical insights into the influence of ChatGPT-facilitated programming on primary learners’ performances: a mixed methods approach”. In: *International Journal of STEM Education*. DOI: [10.1186/s40594-026-00606-1](https://doi.org/10.1186/s40594-026-00606-1) (cit. on pp. 9, 10).

Further Reading

- Hawlitschek, A., S. Berndt & S. Schulz (2022). “Empirical research on pair programming in higher education: a literature review”. In: *Computer Science Education*. DOI: [10.1080/08993408.2022.2039504](https://doi.org/10.1080/08993408.2022.2039504).
- Kohnke, L., D. Zou & H. Xie (2025). “Microlearning and generative AI for pre-service teacher education: a qualitative case study”. In: *Education and Information Technologies*. DOI: [10.1007/s10639-025-13606-5](https://doi.org/10.1007/s10639-025-13606-5).
- Liu, D., G. Fan & L. Pan (2026). “Tool, tutor, or crutch?: A grounded theory of cognitive scaffolding and offloading in AI-assisted programming education”. In: *International Journal of STEM Education*. DOI: [10.1186/s40594-025-00592-w](https://doi.org/10.1186/s40594-025-00592-w).
- Liu, R., C. Zenke, C. Liu, et al. (2024). “Teaching CS50 with AI: Leveraging generative artificial intelligence in computer science education”. In: *Proc. 55th ACM Technical Symposium on Computer Science Education (SIGCSE)*. DOI: [10.1145/3626252.3630938](https://doi.org/10.1145/3626252.3630938).
- Loksa, D., L. Margulieux, B. A. Becker, et al. (2022). “Metacognition and self-regulation in programming education: Theories and exemplars of use”. In: *ACM Transactions on Computing Education*. DOI: [10.1145/3487050](https://doi.org/10.1145/3487050).
- Pawagi, M. & V. Kumar (2024). “Probeable problems for beginner-level programming-with-AI contests”. In: *Proc. ACM Conference on International Computing Education Research (ICER)*. DOI: [10.1145/3632620.3671108](https://doi.org/10.1145/3632620.3671108).
- Sandu, R., E. Gide & M. Elkhodr (2024). “The role and impact of ChatGPT in educational practices: insights from an Australian higher education case study”. In: *Discover Education*. DOI: [10.1007/s44217-024-00126-6](https://doi.org/10.1007/s44217-024-00126-6).